🟢 **3-Tier Application — Real AWS Implementation**

🧠 **First: 3 Tiers ante enti?**
3-tier application ni 3 separate layers ga divide chestam:
1️⃣ **Presentation Layer → Frontend/UI**
2️⃣ **Application Layer → Backend/API**
3️⃣ **Database Layer → Data**
Simple flow:
User → Frontend → Backend API → Database

🌐 **1. Presentation Layer — Frontend**
Suppose React application undi.
Example:
Login Page
Product Page
Cart Page
Checkout Page
Ela deploy chestam?
👉 React application ni build chestam:
npm install
npm run build
👉 Build ayyaka dist/ or build/ files vastayi.
👉 Ee static files ni **Amazon S3** bucket lo store chestam.
👉 Users fast ga access cheyyadaniki **CloudFront** ni S3 mundu place chestam.
Architecture:
User
↓
CloudFront
↓
S3
↓
React Frontend
🔍 **Check**
aws s3 ls
aws cloudfront list-distributions
🛠️ **Verify**
👉 Browser lo CloudFront URL open cheyyali.
👉 React UI properly load avtunda check cheyyali.

⚙️ **2. Application Layer — Backend**
Suppose backend Node.js application.
Example APIs:
/login
/products
/orders
/payment
👉 Backend application ni Docker image ga package cheyyachu.
docker build -t ecommerce-backend .
👉 Image ni ECR ki push chestam.
docker push <ecr-repository>/<image>:latest
👉 Backend ni EC2 instances lo run cheyyachu.
Production lo multiple EC2 instances use chestam.

```
        ┌── EC2 Backend 1
ALB ────┼── EC2 Backend 2
        └── EC2 Backend 3
```

🔍 **Check**

aws ec2 describe-instances
aws elbv2 describe-load-balancers

🛠️ **Verify**

👉 ALB DNS name ki request send cheyyali:
curl http://<ALB-DNS>
👉 Backend response vastunda check cheyyali.

📈 **3. Auto Scaling**

Suppose normal ga 2 EC2 instances unnayi.
Traffic sudden ga increase ayithe:
2 instances
↓
Traffic increases
↓
4 instances
↓
6 instances
AWS Auto Scaling automatically instances increase/decrease cheyyagaladu.
Example:
Minimum = 2
Desired = 2
Maximum = 6

🔍 **Check**

aws autoscaling describe-auto-scaling-groups

🛠️ **Verify**

👉 Load generate chesi CPU increase ayindo observe cheyyali.
👉 New EC2 instances launch ayyaya check cheyyali.

🗄️ **4. Database Layer — RDS**

Backend directly database ki connect avtundi.
Example:
Amazon RDS MySQL
Backend
↓
RDS MySQL
👉 Database ni **private subnet** lo place chestam.
👉 Internet nundi direct database access undakudadhu.
Example:
Internet
❌
↓
Private RDS
Only backend security group ki database access allow chestam.

🔍 **Check**

aws rds describe-db-instances

🛠️ **Verify**

Backend nundi database connection test cheyyali.
Example:
mysql -h <RDS-ENDPOINT> -u <USERNAME> -p

🔐 **5. Security Groups**

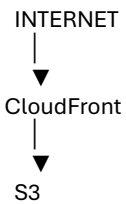
Three layers ki separate security rules maintain cheyyachu.
Frontend
Internet
↓
CloudFront
↓

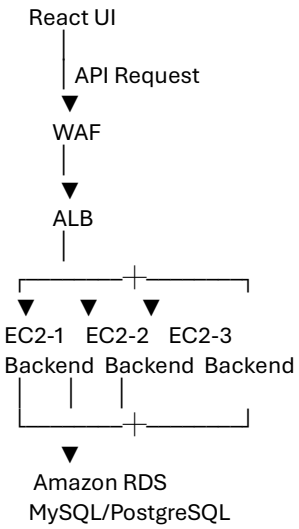
S3
Backend
Internet
↓
ALB : 443
↓
EC2 : 8080
Database
EC2
↓
RDS : 3306
👉 RDS ki **0.0.0.0/0 allow cheyyakudadhu**.

🔒 **6. HTTPS**
User:
https://www.myapp.com
👉 HTTPS certificate kosam **AWS Certificate Manager (ACM)** use chestam.
👉 CloudFront/ALB daggara certificate attach chestam.
Flow:
User
↓ HTTPS
CloudFront
↓
S3
API:
Frontend
↓ HTTPS
ALB
↓
Backend

🛡️ **7. WAF**
Application public-facing kabatti **AWS WAF** add cheyyachu.
WAF common attacks ni filter cheyyadaniki use chestam.
Internet
↓
WAF
↓
CloudFront / ALB
↓
Application
Example:
👉 malicious request
👉 suspicious traffic
👉 rate-based attacks
WAF rules batti block cheyyachu.

🔄 **8. Complete Real Architecture**
Important correction: **S3 nundi ALB ki traffic direct ga flow avvadu.**
Frontend browser lo load ayyaka, browser separate ga backend API calls chestundi.
Correct architecture:





🚀 **9. CI/CD Add Chesthe**

Developer code push chestadu:

Developer
↓
GitHub
↓
CI/CD
↓
Build
↓
Test
↓
Docker Build
↓
ECR
↓
Deploy
↓
EC2 / Auto Scaling

Frontend ki:

GitHub
↓
CI/CD
↓
npm build
↓
S3
↓
CloudFront

Backend ki:

GitHub
↓
CI/CD
↓
Docker Build
↓
ECR
↓
EC2

📊 **10. Monitoring**

Production lo **CloudWatch** use chestam.

Monitor:

CPU
Memory
Request Count
HTTP 4xx
HTTP 5xx
Latency
EC2 Health
RDS Health
ALB Health

Example:

CPU > 80%
↓
CloudWatch Alarm
↓
Auto Scaling
↓
New EC2 Instance

🏗️ **11. Terraform tho Create Cheyyali Ante**

Manual ga AWS Console lo anni create cheyyakunda Terraform use cheyyachu.

Terraform create chestundi:

VPC
├— Public Subnets
├— Private Subnets
├— Internet Gateway
├— NAT Gateway
├— Security Groups
├— ALB
├— Auto Scaling Group
├— EC2
├— RDS
├— S3
└— CloudFront

Command:

terraform init
terraform plan
terraform apply

Verify

terraform output

🔥 **Real Project Execution Order**

Nuvvu actual project chesthe **ee order** follow avvu:

1. VPC create
↓
2. Public + Private Subnets
↓
3. Security Groups
↓
4. RDS create
↓
5. ALB create
↓
6. EC2 + Auto Scaling
↓

7. Backend deploy
   ↓
8. S3 create
   ↓
9. Frontend build + upload
   ↓
10. CloudFront
   ↓
11. Route 53 Domain
   ↓
12. ACM HTTPS
   ↓
13. WAF
   ↓
14. CI/CD
   ↓
15. CloudWatch Monitoring

🎯 **Interview lo "How will you build a 3-tier application?" ante**

"I would create a VPC with public and private subnets. For the presentation layer, I would host the static frontend in S3 and serve it through CloudFront. For the application layer, I would deploy the backend on EC2 instances behind an Application Load Balancer and use Auto Scaling for high availability and traffic handling. For the database layer, I would use Amazon RDS in private subnets with Multi-AZ for high availability. I would secure the architecture using IAM, Security Groups, HTTPS, and optionally WAF. I would use Terraform for infrastructure provisioning, CI/CD for deployments, and CloudWatch for monitoring."